



Implementasi Algoritma Dijkstra Untuk Pemilihan Rute Terpendek Pengambilan Sampah Pada Sistem Pemantauan Edge Computing Smart Trash Bin

^{[1]*}Linda, ^[2]Syamsul Bahri, ^[3]Kasliono

^{[1][2][3]} Jurusan Rekayasa Sistem Komputer, Fakultas MIPA, Universitas Tanjungpura, Pontianak, Indonesia

e-mail: ^{[1]*}h1051201013@student.untan.ac.id, ^[2]syamsul.bahri@siskom.untan.ac.id,
^[3]kasliono@siskom.untan.ac.id

Email Penulis Korespondensi: h1051201013@student.untan.ac.id

Submitted: 15/09/2025; Accepted: 24/09/2025; Published: 06/04/2026

Abstrak

Dalam sistem pengambilan sampah berbasis IoT memerlukan perencanaan rute yang efisien untuk meminimalkan jarak tempuh dan waktu perjalanan. Permasalahan yang terjadi adalah rute ditentukan berdasarkan volume sampah tanpa mempertimbangkan jarak sehingga memungkinkan pengambilan sampah dengan rute yang panjang. Oleh karena itu, sistem perlu mengintegrasikan optimasi rute berdasarkan jarak sebagai prioritas utama, selain volume sampah. Penelitian ini mengimplementasikan algoritma Dijkstra untuk menentukan rute terpendek pengambilan sampah pada sistem pemantauan edge computing. Sistem terdiri dari 12 simpul, meliputi 5 lokasi edge tempat sampah, 6 simpul persimpangan, dan 1 titik awal petugas. Data jarak antar simpul direpresentasikan dalam bentuk graf sesuai kondisi lapangan, sedangkan volume sampah diperoleh secara real-time melalui sensor ultrasonik pada setiap lokasi edge. Data dikirim dari node ke server edge dan diteruskan ke cloud untuk penyimpanan serta pemrosesan lebih lanjut. Hasil pengujian menunjukkan waktu eksekusi rata-rata algoritma sebesar 0,021 ms untuk satu lokasi penuh, 0,00705 ms untuk dua lokasi penuh, 0,0121 ms untuk tiga lokasi penuh, 0,00695 ms untuk empat lokasi penuh, dan 0,00625 ms untuk lima lokasi penuh. Pengujian akurasi membuktikan algoritma mampu menentukan rute terpendek. Pengujian delay pengiriman data menunjukkan hasil yang baik, dengan rata-rata delay 172,6 ms dari node ke edge dan 148,8 ms dari edge ke cloud, memenuhi standar TIPHON.

Kata kunci— Pengambilan Sampah, Algoritma Dijkstra, Rute Terpendek, Edge Computing.

1. PENDAHULUAN

Smart Trash Bin adalah tempat sampah pintar yang mampu mempermudah pekerjaan petugas kebersihan [1]. Dalam hal komunikasi data, *smart trash bin* dapat memanfaatkan teknologi *edge computing* untuk meminimalisir keterlambatan dalam respon data dan ketergantungan pada koneksi internet. *Edge computing* adalah sistem komputasi yang memungkinkan pemrosesan dan penyimpanan data dilakukan secara lokal, dekat dengan perangkat *IoT* dan sumber data. Sistem ini berada di antara perangkat *IoT* dan pusat data, sehingga memungkinkan pengolahan data dan pengambilan keputusan dilakukan langsung diperangkat *edge* atau server terdekat tanpa

perlu mengirim data ke *cloud* terlebih dahulu [2].

Smart trash bin memiliki fitur untuk prioritas pengambilan sampah. Urutan prioritas diatur berdasarkan banyak volume sampah. Sistem ini tidak memprioritaskan jarak sebagai prioritas utama, sehingga memungkinkan pengambilan sampah dengan rute yang panjang. Untuk mengatasi kelemahan ini, sistem perlu mengintegrasikan optimasi rute berdasarkan jarak sebagai prioritas utama, selain volume sampah.

Algoritma Dijkstra merupakan salah satu algoritma yang digunakan untuk menemukan rute terpendek. Algoritma ini termasuk dalam kategori algoritma pencarian graf yang mampu

menyelesaikan masalah pencarian rute terpendek dengan satu sumber pada sebuah graf yang tidak memiliki bobot sisi negatif, dan menghasilkan alur lintasan terpendek [3].

Penelitian berjudul “Implementasi Algoritma Dijkstra untuk Menentukan Rute Terpendek Menuju Pelayanan Kesehatan” menerapkan algoritma Dijkstra untuk menentukan rute terpendek ke pelayanan kesehatan. Algoritma ini digunakan untuk mencari jalur optimal berdasarkan bobot terkecil dari satu titik ke titik lainnya. Hasilnya menunjukkan bahwa Dijkstra mampu memilih rute dengan bobot minimum secara efektif [4]. Pada penelitian ini hanya berfokus pada penerapan algoritma Dijkstra untuk jalur terpendek secara umum, sedangkan penelitian yang akan dilakukan adalah mengembangkannya pada sistem IoT berbasis edge computing untuk pengambilan sampah dengan mempertimbangkan jarak, volume sampah real-time, serta kinerja sistem.

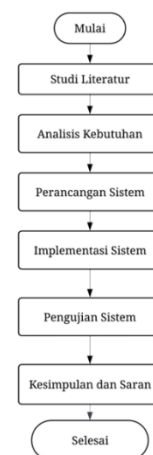
Penelitian berjudul “Perbandingan Algoritma Dijkstra Dan Floyd-Warshall Dalam Menentukan Rute Terpendek Stasiun Gubeng Menuju Wisata Surabaya” membandingkan Algoritma Dijkstra dan Floyd-Warshall untuk menemukan rute terpendek. Hasil penelitian menunjukkan bahwa keduanya menghasilkan bobot minimum yang sama, namun Dijkstra lebih efisien dalam perhitungan. Rute terpendek yang diperoleh adalah menuju Jalan Taman Apsari dengan jarak 1,2 km [5]. Penelitian ini berfokus pada perbandingan 2 algoritma yang berbeda, sedangkan penelitian yang akan dilakukan berfokus pada pengimplementasian algoritma dijkstra pada sistem pemantauan tempat sampah.

Penelitian berjudul “Sistem Pemantauan Tempat Sampah menggunakan Pemodelan *Edge Computing*” menerapkan pemodelan edge computing. Edge server digunakan untuk memproses data secara lokal sehingga ketergantungan pada koneksi internet dapat dikurangi. Hasil pengujian QoS menunjukkan throughput rata-rata 1338 bps dan delay 115 ms yang termasuk kategori sangat baik [6]. Penelitian ini berfokus pada efisiensi jaringan dengan memanfaatkan *edge computing* pada sistem pemantauan tempat sampah, sedangkan penelitian yang akan dilakukan berfokus pada implementasi algoritma Dijkstra untuk efisiensi rute pada sistem pemantauan *edge computing*.

Berdasarkan penelitian yang dilakukan sebelumnya, maka diangkatlah judul “Implementasi Algoritma Dijkstra untuk Pemilihan Rute Terpendek Pengambilan Sampah pada Sistem Pemantauan Edge Computing Smart Trash Bin Berbasis Website”. Penelitian ini bertujuan untuk menentukan rute terpendek pengambilan sampah dengan mengimplementasikan algoritma Dijkstra berdasarkan jarak dan volume sampah. Sistem dirancang untuk memantau volume sampah secara *real-time* melalui pendekatan *edge computing*. Diharapkan sistem ini dapat menghasilkan waktu pemrosesan algoritma yang cepat dan *delay* pengiriman data yang rendah, sehingga mendukung pemantauan sistem secara *real-time*.

2. METODOLOGI PENELITIAN

Metode penelitian digunakan untuk memastikan setiap tahapan yang dilakukan, sehingga hasil yang dicapai sesuai dengan tujuan yang telah diharapkan. Adapun diagram alir tahapan metode penelitian dapat dilihat pada Gambar 1.



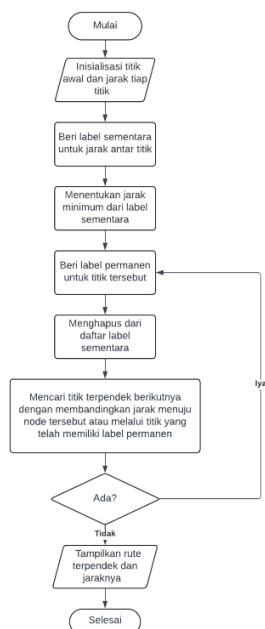
Gambar 1 Diagram Alir Penelitian

Pada tahap studi literatur, dilakukan proses pengumpulan informasi terkait sistem pemantauan edge computing, implementasi algoritma Dijkstra, perangkat IoT, dan cara kerja sensor ultrasonik. Literatur berasal dari buku dan jurnal penelitian yang digunakan sebagai acuan perancangan sistem dan pembandingan hasil penelitian.

Tahap kedua merupakan analisis kebutuhan. Adapun kebutuhan dari penelitian ini, Pertama yaitu Algoritma Dijkstra adalah salah satu algoritma yang efektif dalam

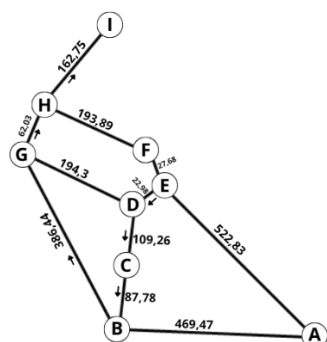
menemukan rute terpendek dari satu lokasi ke lokasi lainnya. Prinsip utama dari algoritma ini adalah mencari titik lokasi dengan melakukan pencarian dua lintasan terpendek. Pada setiap iterasi, jarak dari titik yang diketahui (dari titik awal) diperbarui jika ditemukan titik baru yang memberikan jarak lebih pendek. Syarat dari algoritma ini adalah bobot sisi graf harus non-negatif [7].

Pada setiap tahapannya, algoritma Dijkstra mencari jalur terbaik menuju satu simpul. Artinya, di langkah ke-n, setidaknya sudah ada n simpul yang memiliki jarak terpendek yang telah ditentukan. Tahapan dalam algoritma Dijkstra dapat digambarkan melalui diagram alir [8]. Flowchart algoritma Dijkstra dapat dilihat pada Gambar 2.



Gambar 2 Flowchart Algoritma Dijkstra

Pada penelitian ini akan dilakukan perhitungan algoritma Dijkstra jika tempat sampah pada lokasi edge Konferensi penuh. Graf lokasi edge Konferensi dapat dilihat pada Gambar 3.



Gambar 3 Graf Lokasi Edge Konferensi Penuh

Pada gambar 4 titik awal petugas adalah simpul A dan titik lokasi edge Konferensi adalah simpul I. Langkah-langkah untuk mencari rute terpendek dari lokasi titik awal ke titik lokasi edge Konferensi dengan algoritma Dijkstra sebagai berikut:

Tahap pertama yaitu tahap Inisiasi yaitu menjadikan semua jarak menjadi ∞ , kecuali A = 0. Semua simpul belum dikunjungi. Jarak dari titik awal petugas dapat dilihat pada Tabel 1.

Tabel 1 Rute Awal

Simpul	Jarak dari A (m)	Rute
A	0	A
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-
H	∞	-

Selanjutnya, menghitung jarak graf yang terhubung langsung dengan simpul A. Simpul yang terhubung langsung yaitu simpul B dan simpul E. Jarak dari titik awal petugas ke simpul B adalah 469,47 meter dengan rute A→B, sedangkan jarak dari titik awal ke simpul E adalah 522,83 meter dengan rute A→E.

Selanjutnya dipilih simpul B sebagai simpul dengan jarak terkecil dari simpul A, lalu periksa semua simpul tetangga dari B. Berdasarkan arah graf, simpul B hanya memiliki satu arah keluar, yaitu menuju simpul G dengan bobot jarak sebesar 386.44 m. Total jarak dari A ke G melalui B didapatkan simpul G = 469.47 + 386.44 = 855.91 meter dengan rute A→B→G.

Setelah simpul G di proses maka didapatkan rute A→B→G = 855.91 meter dan simpul G disimpan sementara, lalu algoritma Dijkstra mencari simpul yang belum dikunjungi dengan jarak terkecil selanjutnya yaitu simpul E. . Simpul yang terhubung denngan simpul E yaitu simpul D dan simpul F. Jarak dari simpul E ke D adalah $D = 522.83 + 22.98 = 545.81$ m dan jarak dari simpul E ke F adalah $F = 522.83 + 27.68 = 550.51$ m. Dengan demikian, rute yang terbentuk adalah A→E→D sejauh 545,81 meter dan A→E→F sejauh 550,51 meter.

Rute terpendek berikutnya A→E→D, lalu periksa semua simpul tetangga dari simpul D. Pada graf simpul D terhubung ke simpul E dan simpul C, namun karena simpul E sudah diproses sebelumnya dengan jarak yang lebih

kecil, maka tidak perlu dilakukan pembaruan. Simpul D terhubung ke simpul C dengan jarak $C = 545.81 + 109.26 = 655.07$ m. Dengan demikian, rute dari titik awal A ke simpul C adalah $A \rightarrow E \rightarrow D \rightarrow C$ sejauh 655,07 meter.

Setelah simpul C di proses maka didapatkan rute $A \rightarrow E \rightarrow D \rightarrow C = 655.07$ m dan simpul C disimpan sementara, lalu algoritma Dijkstra mencari simpul yang belum dikunjungi dengan jarak terkecil selanjutnya yaitu simpul F. Simpul yang terhubung dengan simpul F yaitu simpul H dengan jarak dari simpul F ke H adalah $H = 550.51 + 193.89 = 744.40$ m dengan rute $A \rightarrow E \rightarrow F \rightarrow H$.

Selanjutnya, dari hasil pemeriksaan rute diperoleh bahwa jalur $A \rightarrow E \rightarrow D \rightarrow C$ tidak efisien karena terhubung kembali ke simpul B, sehingga rute tersebut diabaikan. Rute terpendek yang dipilih adalah $A \rightarrow E \rightarrow F \rightarrow H$ dengan total jarak 744,40 meter. Dari simpul H langsung menuju simpul I (lokasi tujuan) dengan tambahan jarak 162,75 meter, sehingga total jarak dari A ke I adalah 907,15 meter melalui rute $A \rightarrow E \rightarrow F \rightarrow H \rightarrow I$.

Dari perhitungan yang telah dilakukan pada setiap langkah, maka didapatkan hasil rute terpendek dari titik awal petugas (A) ke titik tujuan lokasi Konferensi (I) yaitu $A \rightarrow E \rightarrow F \rightarrow H \rightarrow I$ dengan total jarak 907.15 m. Adapun rute yang dilalui dan jarak yang dihasilkan dapat dilihat pada Tabel 2.

Tabel 2 Hasil Rute Terpendek ke Lokasi Edge Konferensi (I)

Simpul	Jarak Terpendek dari A (m)	Rute Terpendek
A	0.00	A
B	469.47	$A \rightarrow B$
C	655.07	$A \rightarrow E \rightarrow D \rightarrow C$
D	545.81	$A \rightarrow E \rightarrow D$
E	522.83	$A \rightarrow E$
F	550.51	$A \rightarrow E \rightarrow F$
G	855.91	$A \rightarrow B \rightarrow G$
H	744.40	$A \rightarrow E \rightarrow F \rightarrow H$
I	907.15	$A \rightarrow E \rightarrow F \rightarrow H \rightarrow I$

Kedua, *Edge Computing* adalah bentuk komputasi terdistribusi yang melibatkan berbagai teknologi. Tren ini semakin berkembang dalam bidang *big data* karena memindahkan beban kerja komputasi lebih dekat ke pengguna, yang mengurangi *latensi*, penggunaan *bandwidth*, dan *overhead* pada pusat data terpusat [9]. Teknologi ini menyebarkan sumber daya komputasi dan

penyimpanan di tepi jaringan, lebih dekat ke sensor atau sumber data [10].

Ketiga, NodeMCU ESP32 sebagai mikrokontroler karena mendukung kebutuhan pemrosesan dan konektivitas dalam aplikasi IoT serta dilengkapi dengan Bluetooth 4.0 low *energy* dan WiFi [11]. Adapun gambar ESP32 dapat dilihat pada Gambar 4.



Gambar 4 NodeMCU ESP32

Keempat, Sensor Ultrasonik HC-SR04 digunakan untuk mengukur ketinggian volume sampah. Cara kerjanya mengirim pulsa ultrasonik 40 KHz, lalu menghitung waktu pantulan echo dalam mikrodetik [12]. Adapun gambar sensor ultrasonik HC-SR04 dapat dilihat pada Gambar 5.



Gambar 5 Sensor Ultrasonik HC-SR04

Kelima, Arduino IDE adalah perangkat lunak yang digunakan untuk menulis, mengedit, mengunggah, dan memantau kode program pada mikrokontroler. Dibuat dengan Java dan didukung library C/C++, Arduino IDE memudahkan proses input/output serta debugging. Pada penelitian ini, arduino ide digunakan sebagai sarana untuk menuliskna kode ke dalam board mikrokontroller ESP32 [13].

Keenam, yaitu *Server* adalah bagian paling penting dalam sebuah jaringan karena server merupakan pusat dari berbagai data dan aplikasi. Dalam sistem IoT, server berfungsi sebagai penghubung antara perangkat dan aplikasi layanan [14]. Pada penelitian ini, server digunakan untuk mengolah dan menyimpan data dari node serta menampilkan data melalui antarmuka web.

Ketujuh, Protokol HTTP adalah protokol yang memungkinkan komunikasi antara klien dan server. HTTP menggunakan tiga jenis pesan: GET untuk mengambil informasi, POST untuk mengirim data ke server, dan PUT untuk mengunggah data. HTTP juga mendukung operasi CRUD (Create, Read, Update, Delete) pada data yang tersimpan di server. [15].

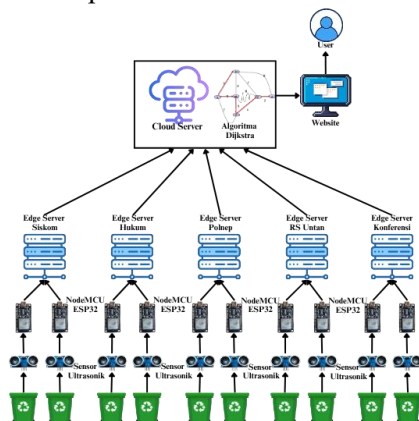
Selanjutnya, *Delay* (latensi), merupakan

total waktu yang diperlukan oleh sebuah paket data selama proses pengiriman dari titik asal ke titik tujuan. *Delay* dapat dipengaruhi oleh beberapa faktor seperti jarak tempuh, jenis media transmisi, kemacetan jaringan (congestion), maupun lamanya waktu pemrosesan. Tabel kategori *delay* berdasarkan standar TIPPHON dapat dilihat pada Tabel 3.

Tabel 3 Kategori *Delay* [16]

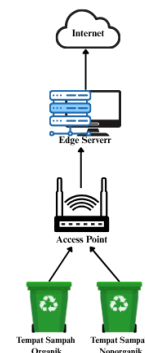
Kategori <i>Delay</i>	<i>Delay</i>	Indeks
Sangat Baik	<150 ms	4
Baik	150 s/d 300 ms	3
Sedang	300 s/d 450 ms	2
Buruk	>450 ms	1

Setelah dilakukan analisis kebutuhan, langkah penelitian selanjutnya adalah perancangan sistem. Perancangan arsitektur sistem pada penelitian ini meliputi perancangan perangkat keras dan perangkat lunak. Perancangan perangkat keras meliputi perancangan NodeMCU ESP32 dan sensor ultrasonik HC-SR04. Node yang digunakan sebanyak sepuluh buah dan masing-masing node terdapat satu buah sensor ultrasonik. Setiap node dan sensor diletakkan pada sepuluh tempat sampah. Terdapat lima lokasi edge tempat sampah dengan masing-masing lokasi terdapat dua tempat sampah. Data volume sampah yang dihasilkan sensor dikumpulkan terlebih dahulu di ESP32. Kemudian data tersebut dikirimkan ke *edge server*, data dari *edge server* disimpan dan dikirimkan ke *cloud server* untuk diolah dan volume sampah dengan kategori penuh (70%-100%) akan dipetakan lokasinya di *maps* dan menentukan rute terpendek menggunakan algoritma Dijkstra, kemudian hasil perhitungan rute terpendek akan ditampilkan di *website maps* untuk memudahkan petugas dalam mengambil sampah. Adapun arsitektur sistem secara umum dapat dilihat pada Gambar 6.



Gambar 6 Arsitektur Sistem

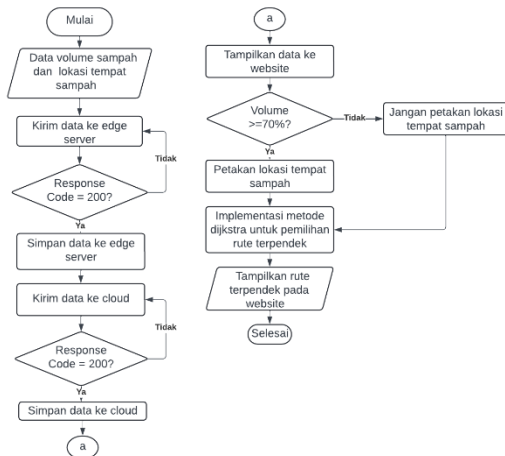
Perancangan sistem *edge computing* proses pengolahan data dilakukan didekat sumber data atau perangkat IoT. Setiap tempat sampah dilengkapi dengan sensor untuk mengukur volume sampah serta NodeMCU ESP32 sebagai mikrokontroler yang mengelola dan mengirimkan data sensor. Data yang dikumpulkan dari sensor kemudian dikirim secara nirkabel melalui *Access Point* (AP). Melalui koneksi *Wi-Fi* yang disediakan oleh *Access Point*, data tersebut diteruskan ke *edge server* yang berperan sebagai pusat pemrosesan awal. Setelah selesai diproses, data dapat dikirim ke *cloud server* melalui koneksi internet untuk diproses lebih lanjut. Adapun perancangan sistem *edge computing* dapat dilihat pada Gambar 7.



Gambar 7 Perancangan Sistem *Edge Computing*

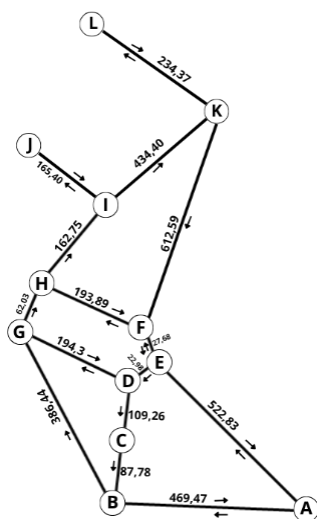
Perancangan perangkat lunak, pertama yaitu perancangan keseluruhan sistem, alur yang dirancang dalam penelitian ini memanfaatkan algoritma Dijkstra untuk memilih rute terpendek pengambilan sampah. Proses dimulai dengan pengambilan data sensor volume sampah dari setiap lokasi tempat sampah yang oleh NodeMCU ESP32 kemudian dikirim ke *edge server*. Data yang dikirimkan ke *edge server* akan diproses dan disimpan sementara sebelum dikirim ke *cloud server*. Kemudian data volume sampah dikirim ke *cloud server*, jika ada kendala saat pengiriman data ke *cloud server* maka data akan disimpan dengan status '*pending*' dan akan dikirim ulang ke *cloud* saat masalah koneksi teratasi, data yang sudah berhasil dikirim ke *cloud* maka akan tersimpan dan diproses oleh *cloud*. Kemudian data tersebut akan ditampilkan ke *website*, jika data volume sampah dari salah satu lokasi penuh, maka lokasi tempat sampah tersebut dipetakan dan jika volume sampah belum penuh maka lokasi tempat sampah tersebut tidak akan dipetakan. Lokasi tempat sampah yang

dipetakan akan ditampilkan di *maps* untuk pemilihan rute terpendek pengambilan sampah menggunakan algoritma Dijkstra. Diagram alir sistem dapat dilihat pada Gambar 8.



Gambar 8 Flowchart Keseluruhan Sistem

Kedua, Perancangan Graf Lokasi *Edge* digunakan sebagai dasar dalam pencarian rute terpendek pengambilan sampah dari satu titik ke titik lainnya. Simpul pada graf berupa huruf abjad dari A-L, dimana A adalah titik awal petugas, B untuk simpang 1, C untuk lokasi edge rs untan, D untuk simpang 5, E untuk lokasi edge hukum, F untuk simpang 4, G untuk simpang 2, H untuk simpang 3, I untuk lokasi edge konferensi, J untuk lokasi edge siskom, K untuk simpang 6, dan L untuk lokasi edge Polnep. Pada graf, setiap lokasi tempat sampah atau simpang jalan digambarkan sebagai simpul, sedangkan jalur yang menghubungkan antar lokasi digambarkan sebagai sisi (*edge*) yang memiliki bobot berupa jarak tempuh. Adapun graf rute lokasi tempat sampah dapat dilihat pada Gambar 9.



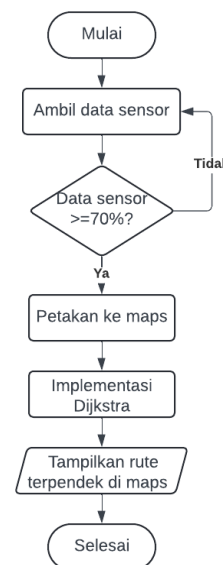
Gambar 9 Graf Lokasi Edge Tempat Sampah

Jarak antar simpul didapat dari *Google Maps* yang dilakukan secara manual dengan cara ukur jarak. Adapun jarak antara simpul yang terhubung dapat dilihat pada Tabel 4.

Tabel 4 Jarak Antar Lokasi

Dari	Ke	Jarak (m)
Titik awal petugas (A)	Simpang 1 (B)	469,47
Simpang 1 (B)	Simpang 2 (G)	384,66
Simpang 2 (G)	Simpang 5 (D)	194,30
Simpang 2 (G)	Simpang 3 (H)	62,03
Simpang 3 (H)	Simpang 4 (F)	193,89
Simpang 4 (F)	Hukum (E)	27,68
Hukum (E)	Simpang 5 (D)	22,98
Simpang 5 (D)	RS Untan (C)	109,26
RS Untan (C)	Simpang 1 (B)	87,78
Simpang 3 (H)	Konferensi (I)	162,75
Konferensi (I)	Siskom (J)	165,40
Konferensi (I)	Simpang 6 (K)	434,40
Simpang 6 (K)	Polnep (L)	234,37
Simpang 6 (K)	Simpang 4 (F)	612,59
Titik awal petugas (A)	Hukum (E)	522,83

Ketiga, Perancangan Implementasi Algoritma Dijkstra pada pengambilan sampah, proses dimulai dari sistem mengambil data volume sampah pada masing-masing *edge*. Jika volume sampah tersebut 70%-100%, maka lokasi tempat sampah tersebut akan dipetakan ke dalam *maps*. Lokasi yang dipetakan ini akan dicari rute pengambilannya menggunakan algoritma Dijkstra. Setelah rutenya didapatkan, maka sistem akan menampilkan rute ini di website yang telah dirancang sebelumnya. Flowchart implementasi algoritma Dijkstra pada sistem pemantauan *edge computing smart trash bin* dapat dilihat pada Gambar 10.



Gambar 10 Flowchart Implementasi Algoritma Dijkstra

Implementasi sistem mencakup pemasangan dan pengaturan perangkat keras berupa dua node yang terdiri dari ESP32 dan sensor ultrasonik di tiap node. Sensor ultrasonik yang terhubung dengan NodeMCU ESP32 berfungsi untuk mengukur volume sampah di dalam tempat sampah dengan meletakkan sensor diatas tutup tempat sampah. Adapun implementasi perangkat keras dapat dilihat pada Gambar 11.



Gambar 11 Implementasi Perangkat Keras

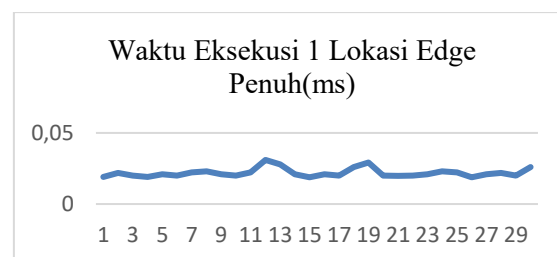
3. HASIL DAN PEMBAHASAN

Implementasi algoritma Dijkstra pada pengambilan sampah ini bertujuan untuk memilih rute terpendek berdasarkan jarak antar lokasi tempat sampah dan volume sampah. Setiap tempat sampah dianggap sebagai simpul (*node*) dalam sebuah graf dan jarak antar titik dihitung berdasarkan jarak nyata yang didapatkan dari *google maps*. Jika volume sampah penuh pada beberapa edge atau lokasi tempat sampah, maka rute akan dipetakan untuk di proses pengambilan sampah dengan menggunakan algoritma Dijkstra sebagai pemilihan rute terpendek. Dan hasil prioritas rute pengambilan sampah serta jarak akan ditampilkan dimaps.

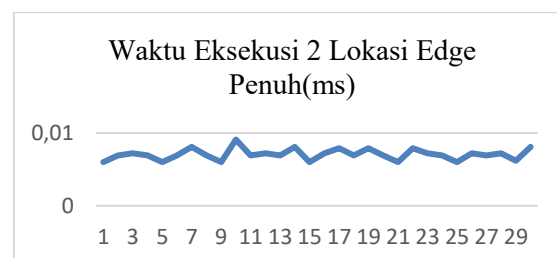
Edge server berfungsi sebagai pemrosesan data yang dekat dengan sumber data. *Edge server* bekerja secara lokal untuk memproses dan mengirimkan data volume tempat sampah ke *cloud server*. *Cloud server* berfungsi sebagai pusat pengolahan data dari masing-masing *edge server*. Pada *cloud server* ini, data lokasi tempat sampah dan volume sampah digunakan untuk menjalankan algoritma Dijkstra, yang menentukan rute terpendek dari lokasi petugas ke lokasi *edge* yang memiliki volume sampah penuh.

Pada penelitian ini dilakukan pengujian waktu eksekusi algoritma Dijkstra, pengujian akurasi algoritma Dijkstra, pengujian *delay* pengiriman data dari node ke *edge server* dan

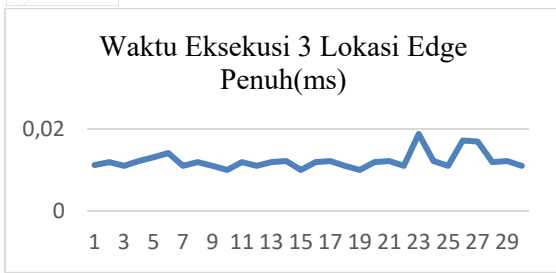
delay pengiriman data dari *edge server* ke *cloud server*. Pengujian pertama, pengujian waktu eksekusi algoritma Dijkstra pada sistem pemantauan *edge computing smart trash bin* dilakukan untuk mengetahui kinerja algoritma Dijkstra dalam menentukan rute terpendek. Pengujian ini dilakukan dengan 5 kali percobaan. Pada satu percobaan digunakan 30 data waktu eksekusi untuk menghitung rata-rata waktu eksekusi algoritma. Hasil pengujian menunjukkan bahwa rata-rata waktu eksekusi yang diperlukan untuk proses algoritma Dijkstra menentukan rute terpendek pada percobaan satu lokasi *edge server* yang sudah penuh adalah 0,021 ms, rata-rata waktu eksekusi yang diperlukan untuk proses algoritma Dijkstra menentukan rute terpendek pada percobaan dua lokasi *edge server* yang sudah penuh adalah 0,00705 ms, rata-rata waktu eksekusi yang diperlukan untuk proses algoritma Dijkstra menentukan rute terpendek pada percobaan tiga lokasi *edge server* yang sudah penuh adalah 0,0121 ms, rata-rata waktu eksekusi yang diperlukan untuk proses algoritma Dijkstra menentukan rute terpendek pada percobaan empat lokasi *edge server* yang sudah penuh adalah ,00695 ms, dan rata-rata waktu eksekusi yang diperlukan untuk proses algoritma Dijkstra menentukan rute terpendek pada percobaan lima lokasi *edge server* yang sudah penuh adalah 0,00625 ms. Grafik waktu eksekusi pada percobaan 1 sampai percobaan 5 dapat dilihat pada Gambar 12 hingga Gambar 16.



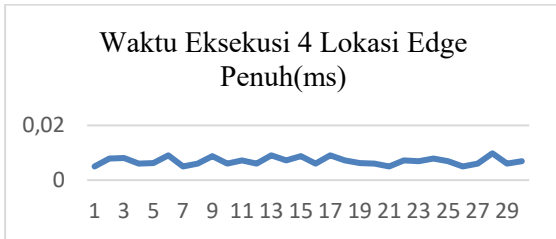
Gambar 12 Grafik Waktu Eksekusi Percobaan 1



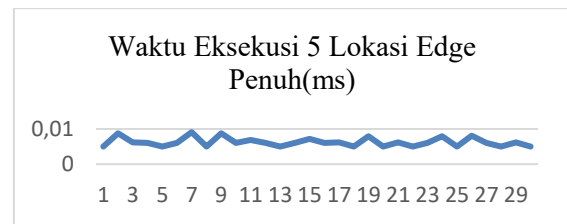
Gambar 13 Grafik Waktu Eksekusi Percobaan 2



Gambar 14 Grafik Waktu Eksekusi Percobaan 3



Gambar 15 Grafik Waktu Eksekusi Percobaan 4



Gambar 16 Grafik Waktu Eksekusi Percobaan 5

Kedua, pengujian akurasi algoritma Dijkstra pada sistem pemantauan *edge computing smart trash bin* dilakukan dengan membandingkan hasil rute dari sistem dan perhitungan manual dalam 5 skenario, mulai dari satu hingga lima lokasi edge yang penuh. Hasilnya, semua rute terpendek dari sistem sama dengan perhitungan manual, menunjukkan bahwa algoritma Dijkstra akurat dalam menentukan rute terpendek. Salah satu pengujian akurasi dari percobaan 1 pada 1 lokasi edge server yang penuh dapat dilihat pada Tabel 5.

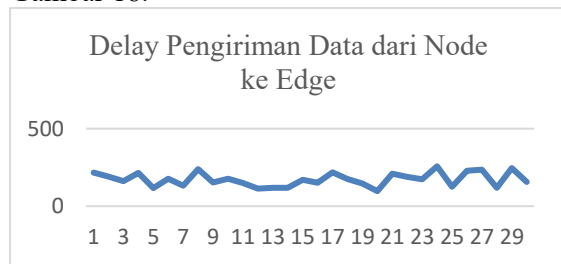
Tabel 5 Pengujian Akurasi Percobaan 1 Lokasi Edge Server yang Penuh

No	Percobaan ke-1	Manual			Rute Terpendek dari Sistem	Keterangan
		Rute yang mungkin	Jarak (m)	Rute Terpendek		
1	Hukum	A→E	522,83	A→E	A→E	Sesuai
		A→B→G→H→F→E	1.137,73			
		A→B→G→H→I→K→F→E	2.153,58			
2	RS Untan	A→B→G→D→C	1.157,69	A→E→D→C	A→E→D→C	Sesuai
		A→B→G→H→F→E→D→C	1.269,97			
		A→E→D→C	655,07			
		A→B→G→H→I→K→F→E→D→C	2.285,82			
3	Konferensi	A→E→F→H→I	907,15	A→E→F→H→I	A→E→F→H→I	Sesuai
		A→B→G→H→I	1.078,91			
		A→E→D→C→B→G→H→I	1.352,29			
		A→E→D→G→H→I	964,89			
4	Siskom	A→B→G→H→I→J	1.244,31	A→E→F→H→I→J	A→E→F→H→I→J	Sesuai
		A→E→D→C→B→G→H→I→J	1.517,69			
		A→E→F→H→I→J	1.072,55			
5	Polnep	A→B→G→H→I→K→L	1.747,68	A→E→D→G→H→I→K→L	A→E→F→H→I→K→L	Sesuai
		A→E→D→C→B→G→H→I→K→L	2.021,06			
		A→E→D→G→H→I→K→L	1.633,66			
		A→E→F→H→I→K→L	1.828,83			

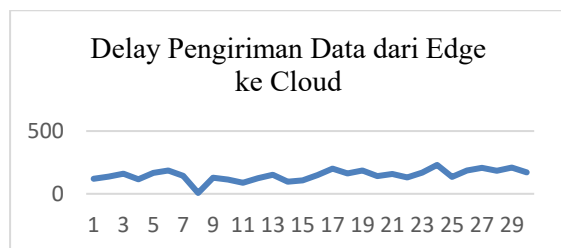
Ketiga, pengujian *delay* dilakukan untuk menilai kecepatan pengiriman data. Pengujian dilakukan sebanyak 30 kali untuk merepresentasikan kondisi rata-rata *delay* dalam sistem. Pengukuran *delay* dilakukan

dengan menggunakan perangkat lunak *wireshark* untuk menangkap paket data yang dikirim dan mencatat waktu pengiriman serta waktu penerimaan pada *cloud server*. Hasil pengujian diperoleh rata-rata *delay* dari node ke

edge server adalah 172,6 ms, masuk kategori baik menurut standar TIPHON, sedangkan dari *edge server* ke *cloud server* sebesar 148,8 ms dan tergolong sangat baik menurut standar TIPHON. Hal ini menunjukkan sistem berjalan optimal dalam pengiriman data. Grafik delay pengiriman data dari node ke *edge* dan dari *edge* ke *cloud* dapat dilihat pada Gambar 17 dan Gambar 18.



Gambar 17 Grafik Delay dari Node ke Edge



Gambar 18 Grafik Delay dari Edge ke Cloud

4. KESIMPULAN

Berdasarkan pembahasan dan hasil pengujian yang telah dilakukan, dapat disimpulkan bahwa hasil pengujian rata-rata waktu eksekusi yang diperlukan algoritma Dijkstra untuk menentukan rute terpendek pada sistem pemantauan *edge computing smart trash bin* pada 5 percobaan dengan masing-masing percobaan terdapat 30 data didapatkan hasil yaitu untuk satu lokasi *edge server* yang sudah penuh sebesar 0,021 ms, dua lokasi *edge server* yang sudah penuh sebesar 0,00705 ms, tiga lokasi *edge server* yang sudah penuh sebesar 0,0121 ms, empat lokasi *edge server* yang sudah penuh sebesar 0,00695 ms, dan lima lokasi *edge server* yang penuh sebesar 0,00625 ms. Selain itu, pengujian akurasi algoritma Dijkstra dengan 5 skenario percobaan menunjukkan bahwa algoritma Dijkstra mampu secara tepat menentukan rute terpendek. Pada pengujian delay pengiriman data yang dilakukan sebanyak 30 kali dengan menggunakan *wireshark* didapatkan hasil *delay* pengiriman data dari node ke *edge server* menunjukkan rata-rata *delay* sebesar 172,6 ms. *Delay* pengiriman data dari *edge server* ke *cloud server* memiliki rata-

rata sebesar 148,8 ms. Dengan demikian, algoritma Dijkstra memiliki performa baik dari segi kecepatan dan akurasi, mendukung pengambilan keputusan rute pengambilan sampah secara tepat dan efisien. Menurut standar TIPHON, *delay* pengiriman data dari node ke *edge server* termasuk kategori baik, sedangkan dari *edge* ke *cloud server* kategori sangat baik, menunjukkan sistem mampu menjaga efisiensi komunikasi data pada kedua arah.

DAFTAR PUSTAKA

- [1] A. P. Dewi, R. Nugraha, S. Sumaryo, F. T. Elektro, U. Telkom, and T. Sampah, "Perancangan dan Implementasi smart trash bin menggunakan metode logika fuzzy," *e-Proceeding Eng.*, vol. 6, no. 2, pp. 2871–2878, 2019.
- [2] Reza, *Sistem pemantauan pada edge computing smart trash bin berbasis website*. 2023.
- [3] A. Cantona, F. Fauziah, and W. Winarsih, "Implementasi Algoritma Dijkstra Pada Pencarian Rute Terpendek ke Museum di Jakarta," *J. Teknol. dan Manaj. Inform.*, vol. 6, no. 1, pp. 27–34, 2020, doi: 10.26905/jtmi.v6i1.3837.
- [4] I. Arthalia Wulandari and P. Sukmasetyan, "Implementasi Algoritma Dijkstra untuk Menentukan Rute Terpendek Menuju Pelayanan Kesehatan," *J. Ilm. Sist. Inf.*, vol. 1, no. 1, pp. 30–37, 2022, doi: 10.24127/jisi.v1i1.1953.
- [5] H. Hendra and Y. F. Riti, "Perbandingan Algoritma Dijkstra Dan Floyd-Warshall Dalam Menentukan Rute Terpendek Stasiun Gubeng Menuju Wisata Surabaya," *JIKA (Jurnal Inform.)*, vol. 6, no. 3, p. 297, 2022, doi: 10.31000/jika.v6i3.6528.
- [6] S. Rokoiye, U. Ristian, And K. Kasliono, "Sistem Pemantauan Tempat Sampah menggunakan Pemodelan Edge Computing," *ELKOMIKA J. Tek. Energi Elektr. Tek. Telekomun. Tek. Elektron.*, vol. 12, no. 2, p. 275, 2024, doi: 10.26760/elkomika.v12i2.275.
- [7] R. T. Kusumastuti *et al.*, "Penerapan Algoritma Dijkstra Dalam Menentukan Rute," vol. 12, no. 1, 2024.
- [8] S. Sunardi, A. Yudhana, and A. A.



- Kadim, “Implementasi Algoritma Dijkstra Untuk Analisis Rute Transportasi Umum Transjogja Berbasis Android,” *J. Sist. Inf. Bisnis*, vol. 9, no. 1, p. 32, 2019, doi: 10.21456/vol9iss1pp32-38.
- [9] D. Dalan, “An Overview of Edge Computing,” *Int. J. Eng. Res. Technol.*, vol. 7, no. 5, pp. 1–4, 2019, [Online]. Available: www.ijert.org
- [10] K. Cao, Y. Liu, G. Meng, and Q. Sun, “An Overview on Edge Computing Research,” *IEEE Access*, vol. 8, pp. 85714–85728, 2020, doi: 10.1109/ACCESS.2020.2991734.
- [11] I. W. Suriana, I. G. A. Setiawan, and I. M. S. Graha, “Rancang Bangun Sistem Pengaman Kotak Dana Punia berbasis Mikrokontroler NodeMCU ESP32 dan Aplikasi Telegram,” *J. Ilm. Telsinas Elektro, Sipil dan Tek. Inf.*, vol. 4, no. 2, pp. 75–84, 2022, doi: 10.38043/telsinas.v4i2.3198.
- [12] R. Sudrajat and F. Rofifah, “Rancang Bangun Sistem Kendali Kipas Angin dengan Sensor Suhu dan Sensor Ultrasonik Berbasis Arduino Uno,” *Remik*, vol. 7, no. 1, pp. 555–564, 2023, doi: 10.33395/remik.v7i1.12082.
- [13] U. Mahanin Tyas, A. Apri Buckhari, P. Studi Pendidikan Teknologi Informasi, and P. Studi Pendidikan Teknologi dan Kejuruan, “Implementasi Aplikasi Arduino Ide Pada Mata Kuliah Sistem Digital,” *Tek. J. Pendidik. Teknol. Inf.*, vol. 1, no. 1, pp. 1–9, 2023.
- [14] C. Rizal, S. Supiyandi, M. Zen, and M. Eka, “Perancangan Server Kantor Desa Tomuan Holbung Berbasis Client Server,” *Bull. Inf. Technol.*, vol. 3, no. 1, pp. 27–33, 2022, doi: 10.47065/bit.v3i1.255.
- [15] A. Ehsan, M. A. M. E. Abuhaliqa, C. Catal, and D. Mishra, “RESTful API Testing Methodologies: Rationale, Challenges, and Solution Directions,” *Appl. Sci.*, vol. 12, no. 9, 2022, doi: 10.3390/app12094369.
- [16] ETSI, “Telecommunications and Internet Protocol Harmonization Over Networks (TIPHON); General aspects of Quality of Service (QoS),” *Etsi Tr 101 329 V2.1.1*, vol. 1, pp. 1–37, 1999.